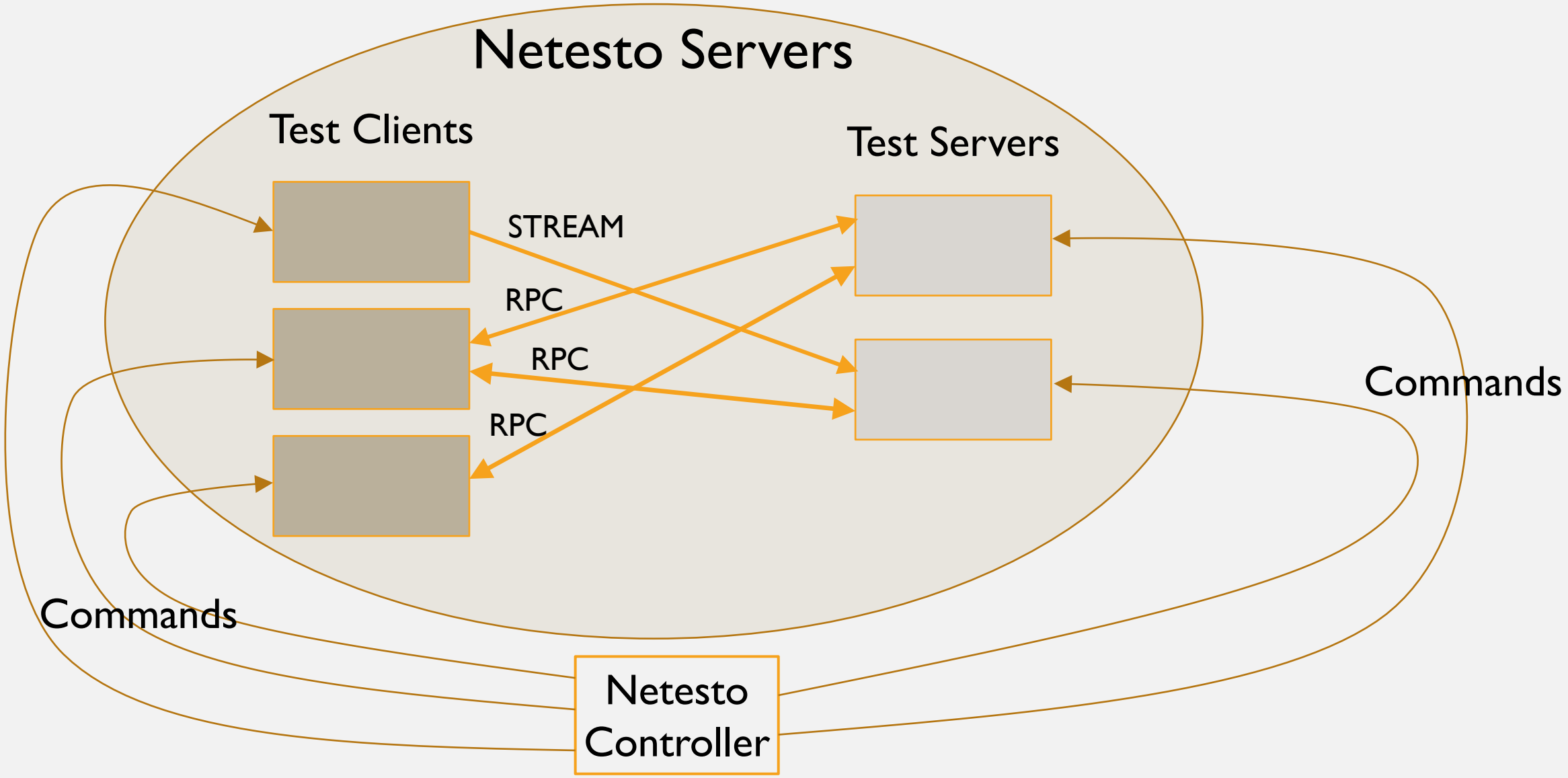
NETESTO
A NETWORK TESTING TOOLKIT

Lawrence Brakmo

Facebook

brakmo@fb.com

NETESTO COMPONENTS AND THEIR INTERACTIONS



OVERVIEW

- Grew organically, no initial grand-design (full of my idiosyncrasies)
- Simple test script language
  - Safer for sharing scripts (vs. having scripts in Python)
- Consists of many programs and scripts divided into 2 directories
  - Local directory: Copied to Netesto Controller
  - Remote directory: Copied to Netesto Servers
- Main program: netesto.py
  - Run remotely to start netesto service
    - ./netesto.py –s [–debug=netesto.log] &
  - Run locally to parse test scripts and control remote machines
    - ./netesto.py [–debug=out.log] < script.tcpCA &

NETESTO CLIENT

- Reads a test script and for each test:

1. Test setup commands (netem for latencies, qdisc for bw and queue size, etc.)
2. Collects initial information from servers (netstat, snmp, ethtool, etc.)
3. Starts the flows and starts flow information collection (netperf, ss, ping)
4. Processes local results into key-value pairs, reads final information
5. Copies test data and results back to the client,
6. Aggregates the individual key-value data into per-test key-value file
7. Adds the key-value data of the test to a csv (comma separated value file) consisting of a line for the aggregate result and additional lines for the individual flows
8. Creates graphs of goodputs, RTTs, cwnds and adds them to HTML page
9. Processes the csv file and creates an HTML page consisting of tables and graphs

SECURITY?

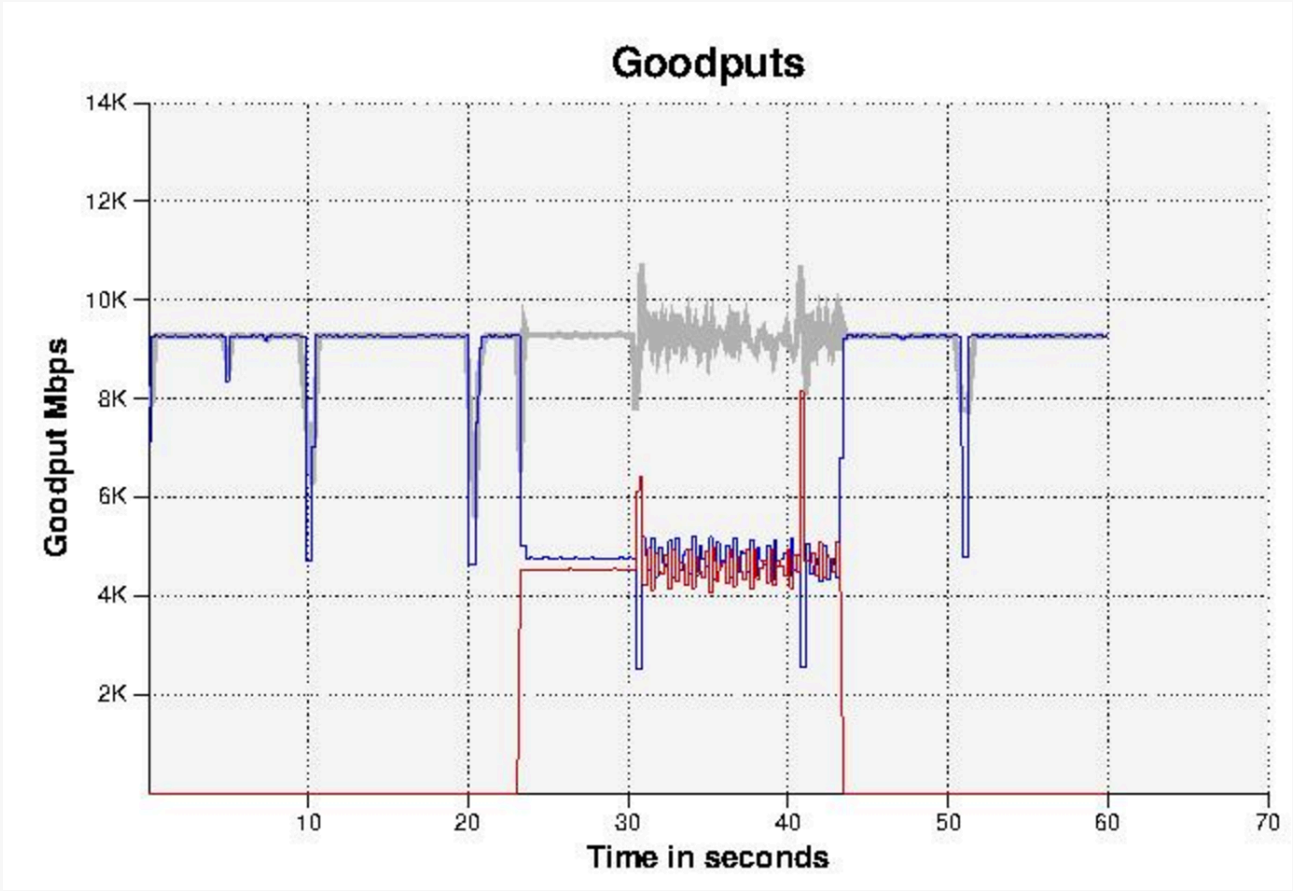
- Limited
- Test Script Language – protects from harmful scripts
- Netesto servers have a whitelist file with IPv6 addresses
  - Will only accept connections from those addresses
- Commands to Netesto servers are well defined and limited
  - Whitelist of allowed sysctl commands
- GET_DATA command only copies Netesto created files
  - Cannot specify what to copy

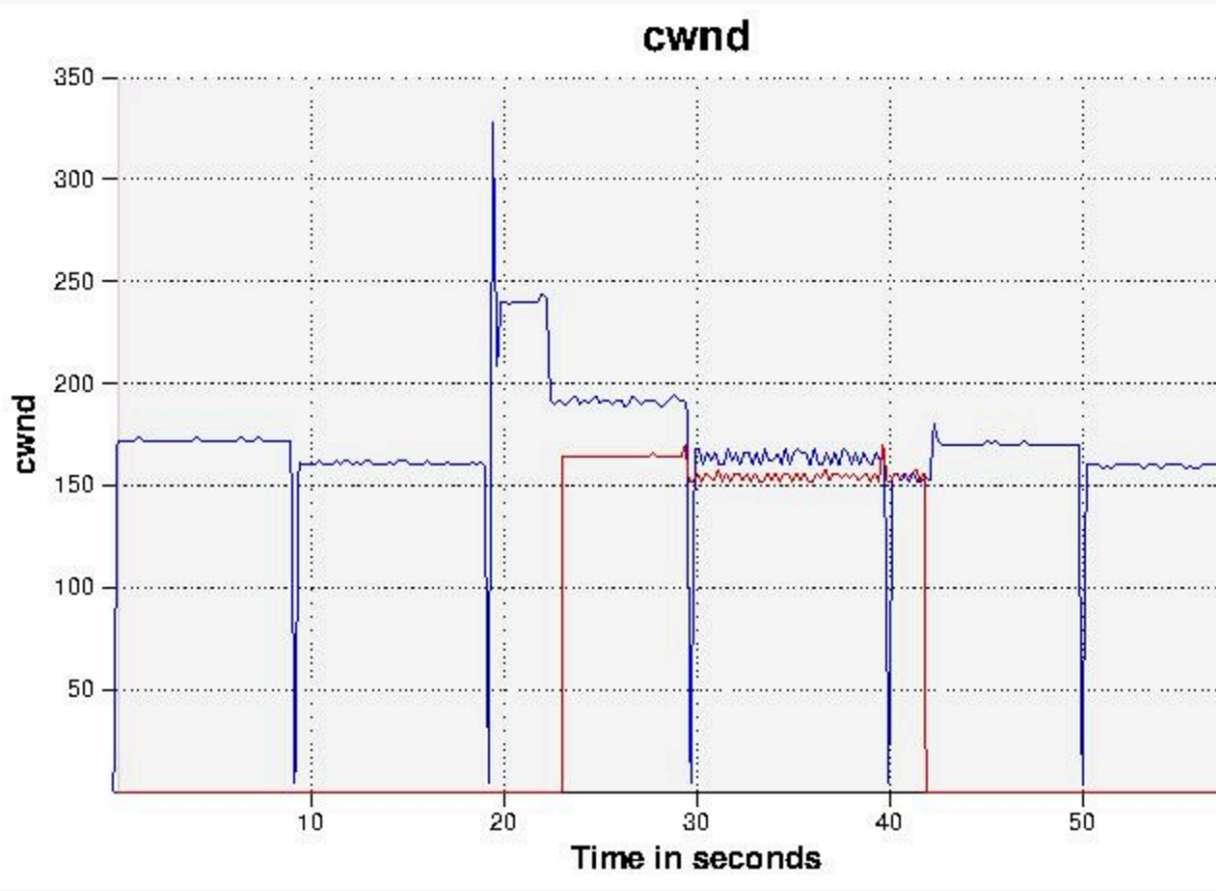
TWO FLOW STREAM TEST SAMPLE

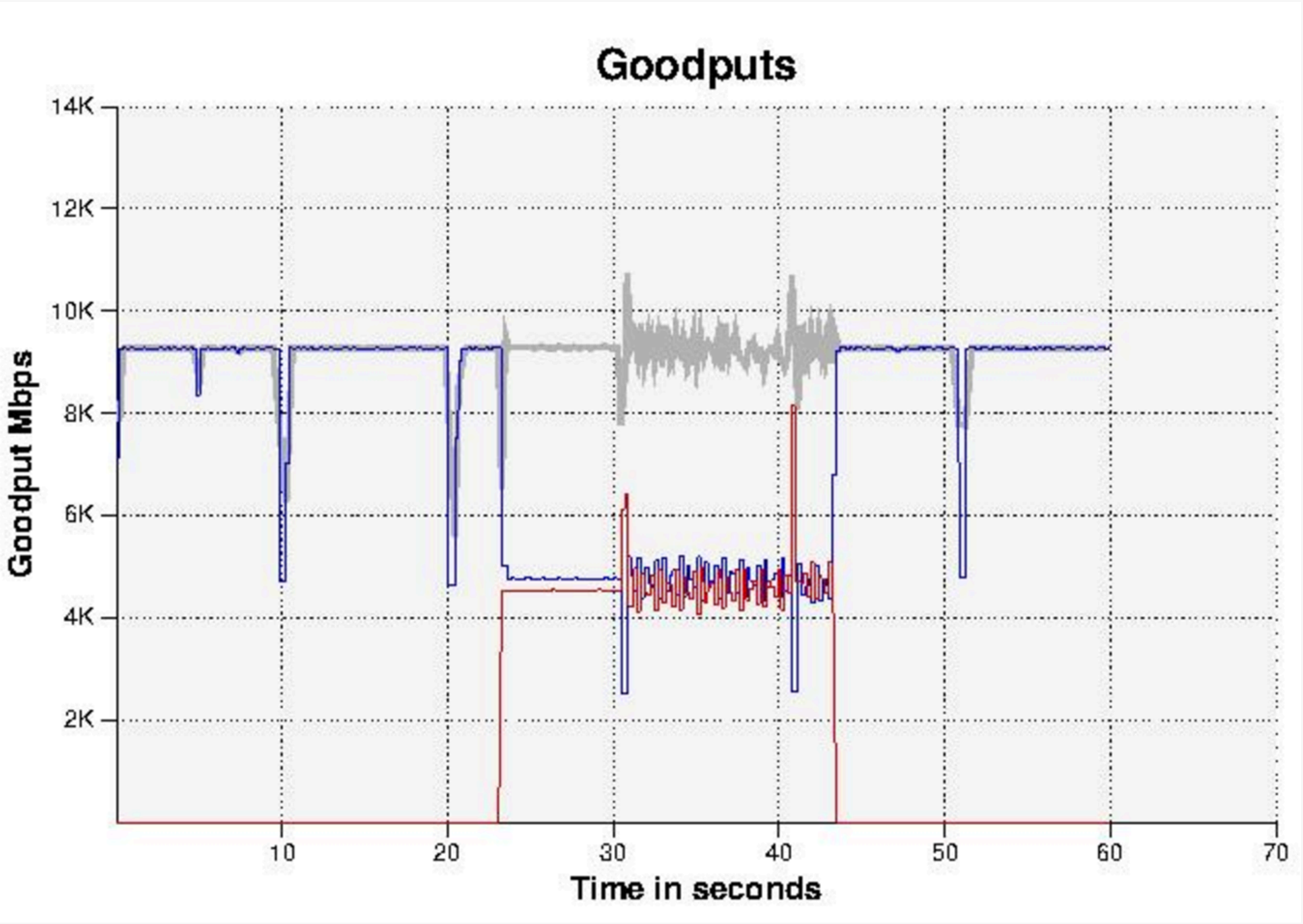
```
HOST_SUFFIX mynetwork.com
SOURCE inlib
SET dur=60
SET_EXP dur2=dur/3
SET_EXP delay2=$dur2+3
SET server=host1
SET client1=host2
SET client2=host3
SET ca=bbr
RUN twoFlowBaseStream,1 server=$server
client1=$client1 client2=$client2 expName=1s2c1fs
ca1=$ca ca2=$ca dur1=$dur dur2=$dur2 delay2=$delay2
```

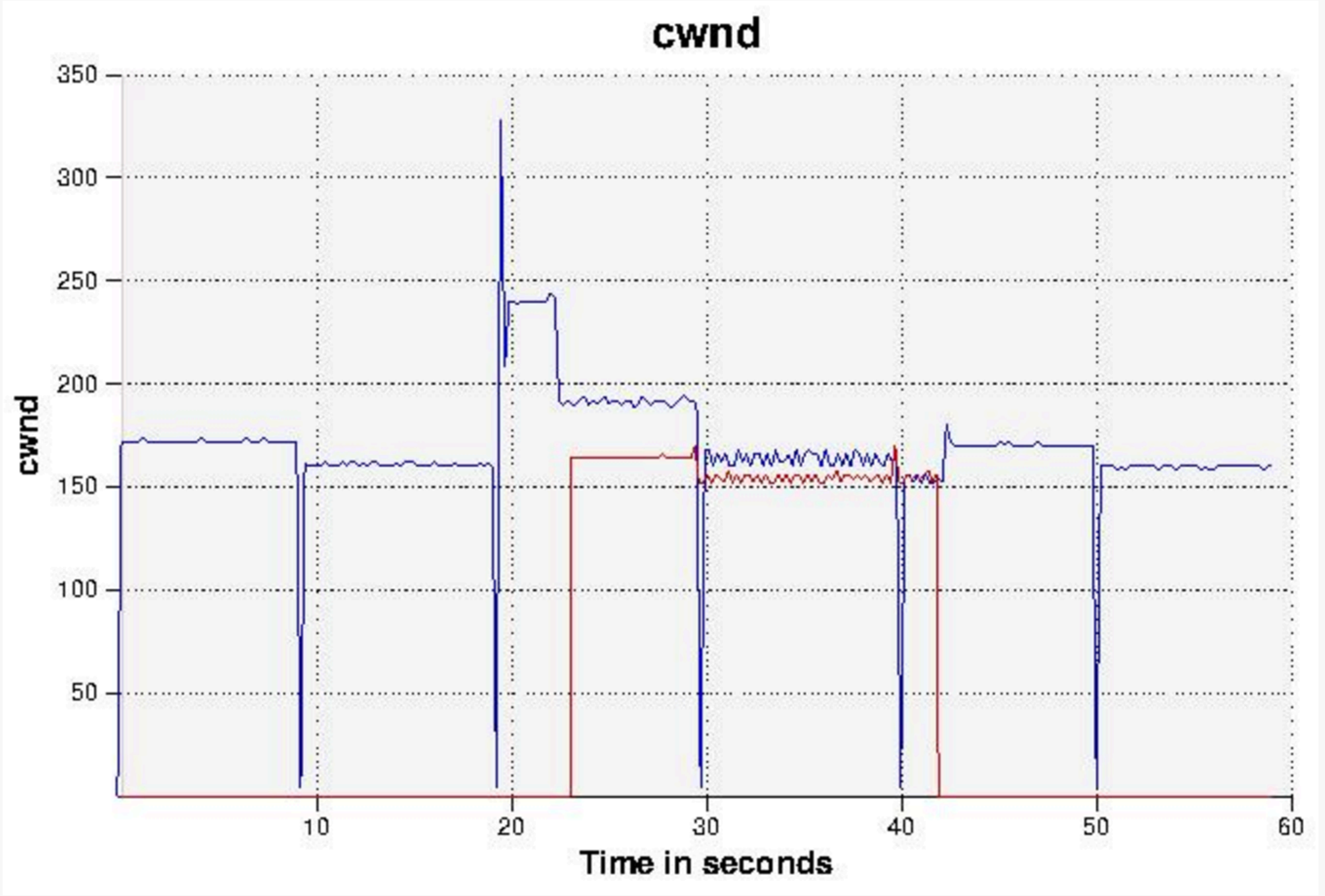
SAMPLE OUTPUT (BBR)

	exp	expName	test	host	server	instances	ca	dur	delay	cwnd	rtt	pingRtt	rate	rateMin	rateMax	retransPkts%
☐	104.0	1s2cmid.vs	TCP_STREAM	kerneltest002+...	kerneltest011	1.0	bbr	60+...	0+...	163.5	295.5	269.0	12229.0	4620.0	7609.0	0.00
☐	104.1	1s2cmid.vs	TCP_STREAM	kerneltest002	kerneltest011	1.0	bbr	60.0	0.0	169.0	246.0	209.0	7609.0	7609.0	7609.0	
☐	104.2	1s2cmid.vs	TCP_STREAM	kerneltest004	kerneltest011	1.0	bbr	20.0	23.0	158.0	345.0	329.0	4620.0	4620.0	4620.0	






Goodputs
14K
12K
10K
8K
6K
4K
2K
Goodput Mbps
10
20
30
40
50
60
70
Time in seconds

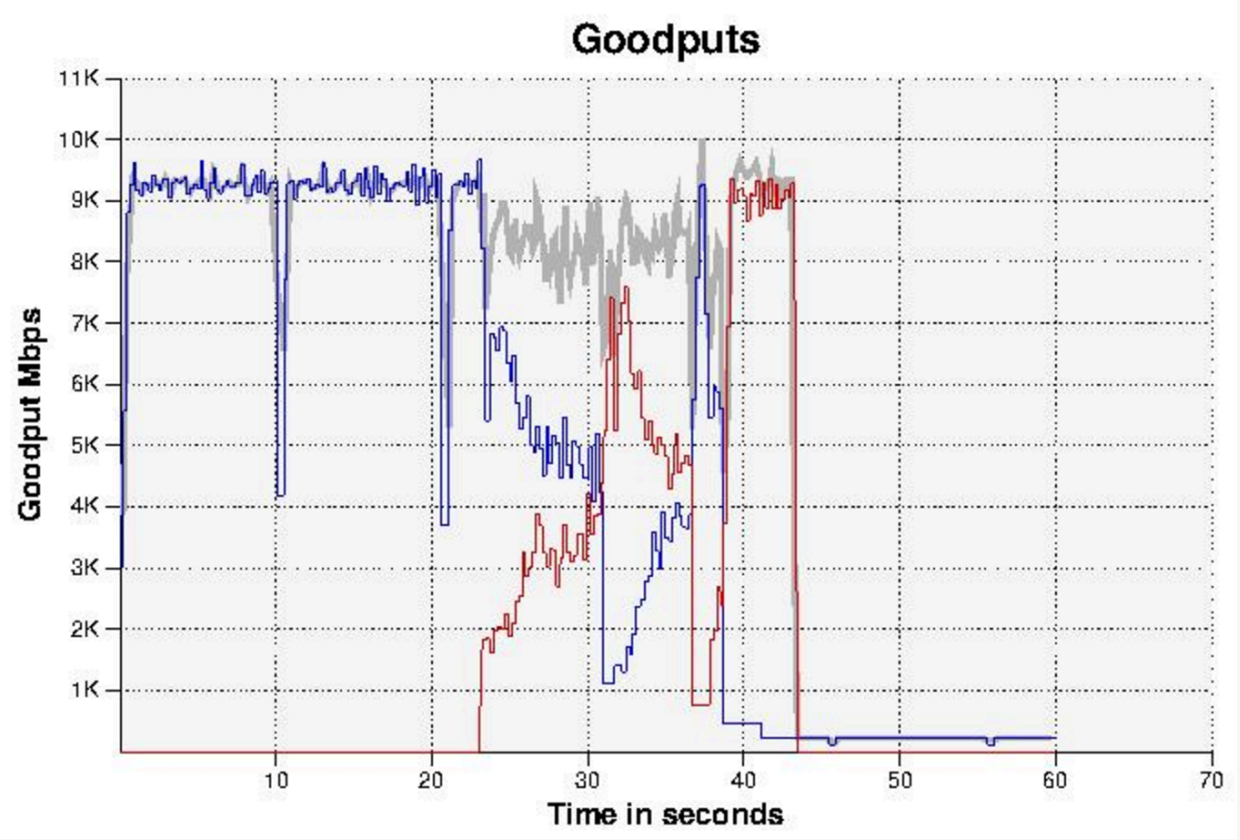

cwnd
350
300
250
200
150
100
50
cwnd
10
20
30
40
50
60
Time in seconds

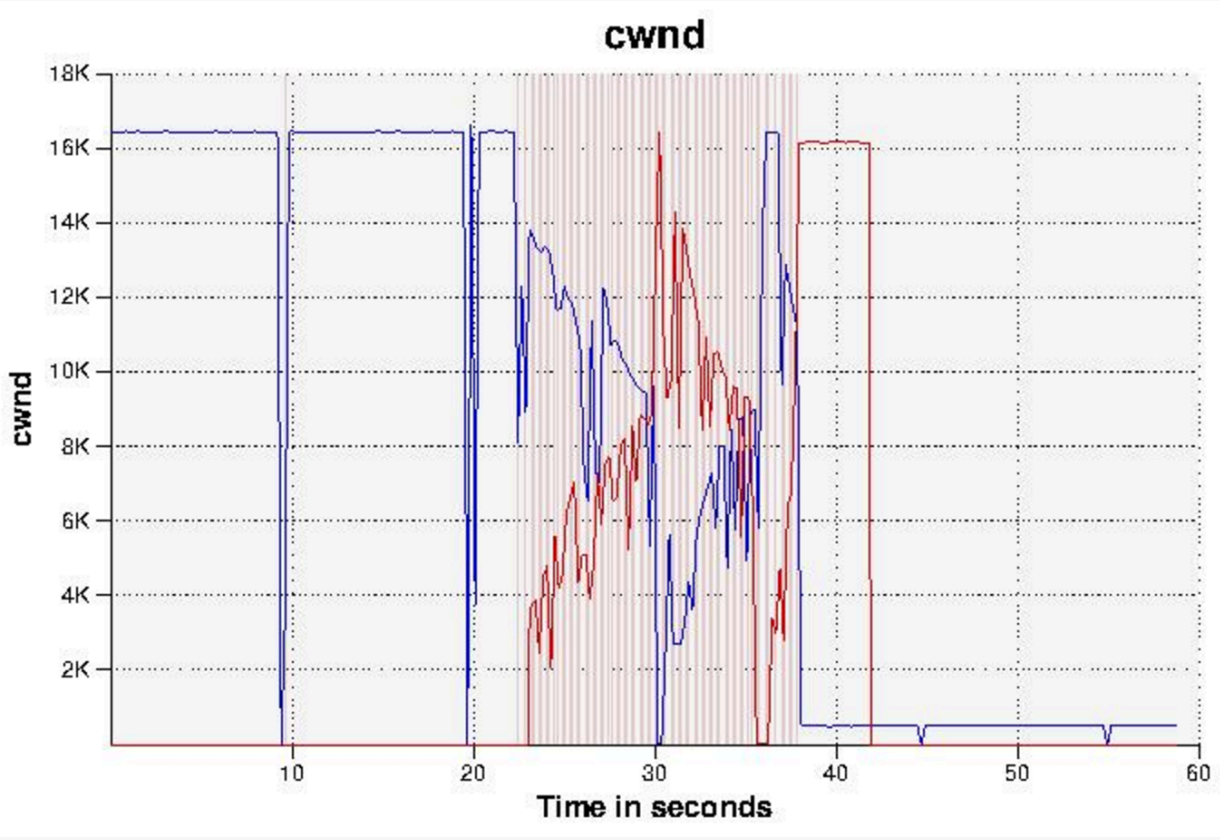
TWOFLOWBASESTREAM MACRO

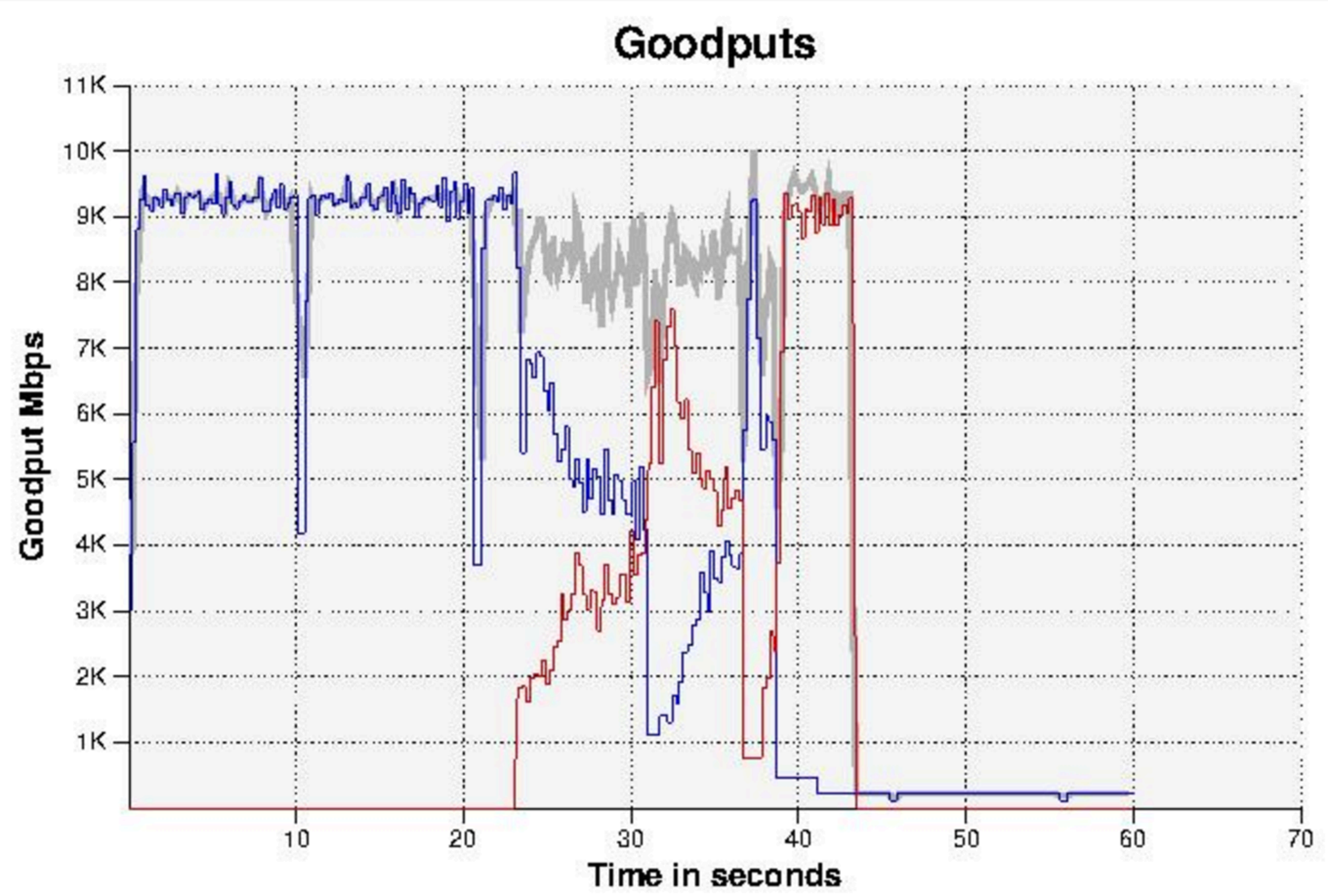
```
BEGIN TwoFlowBaseStream
SET exp=COUNTER
IF_DEF preServer: RUN preServer host=$server
DO_SERVER exp=$exp host=$server \
   expName=$expName order=0 start=1
IF_DEF preClient: RUN preClient \
   host=$client1
IF $tcpDump: DO_TCPDUMP host=$client1 \
   server=$server packets=$tcpDump
DO_CLIENT host=$client1 server=$server \
   ca=$ca1 dur=$dur1 delay=0 stats=1
   instances=$instances test=TCP_STREAM
RAND_WAIT $randDelay
IF_DEF preClient: RUN preClient \
   host=$client2
IF $tcpDump: DO_TCPDUMP host=$client2 \
   server=$server packets=$tcpDump
DO_CLIENT host=$client2 server=$server \
   ca=$ca2 dur=$dur2 delay=$delay2 \
   instances=$instances test=TCP_STREAM\
   stats=1
WAIT $dur1
WAIT 10
DO_SERVER host=$server order=1
WAIT 5
GET_DATA host=$client1
GET_DATA host=$client2
GET_DATA host=$server
WAIT 10
PROCESS_EXP
WAIT 15
END TwoFlowBaseStream
```

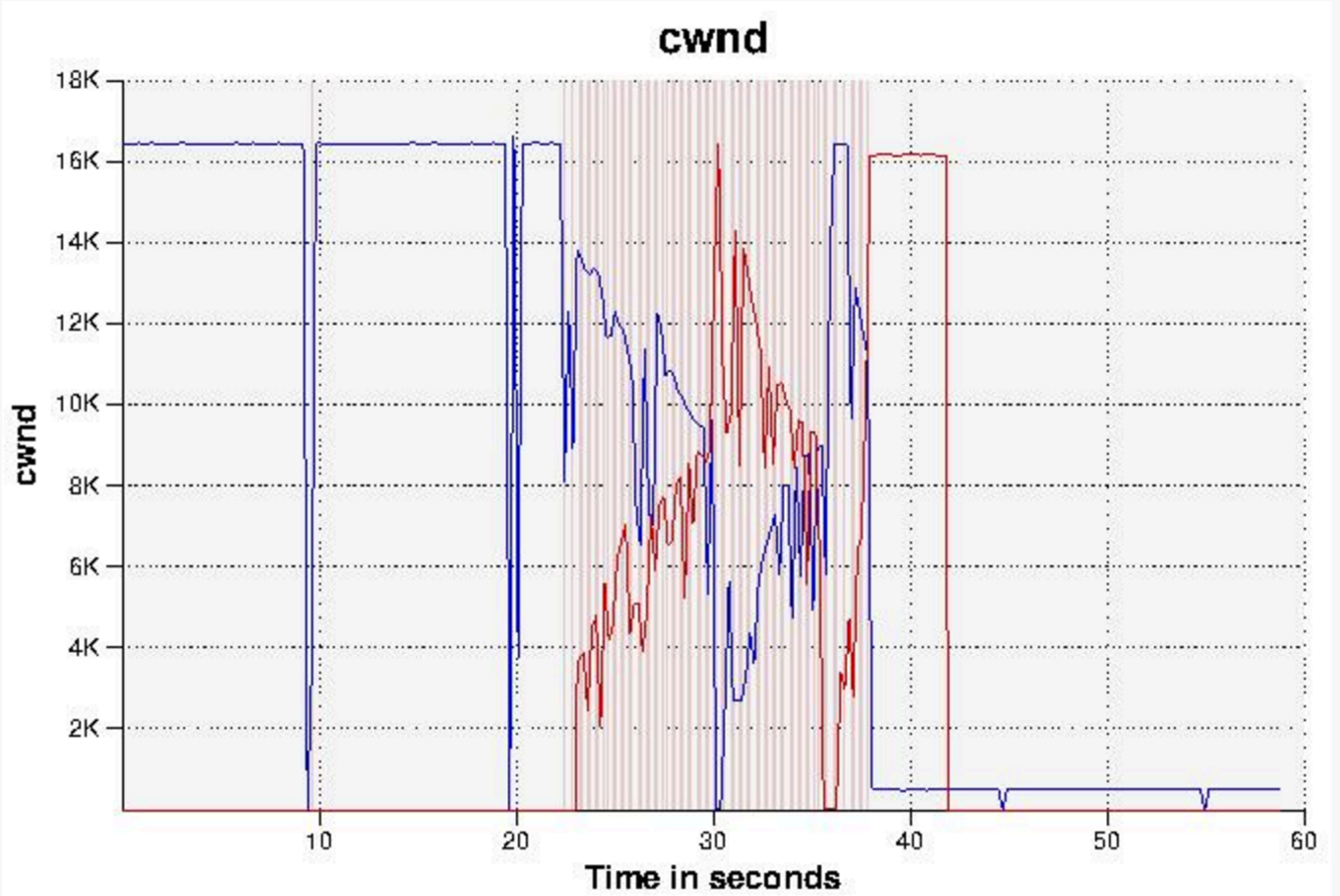
2 FLOWS WITH 10MS RTT, 10GBPS (BBR)

	exp	expName	test	host	server	instances	ca	dur	delay	cwnd	netem	rtt	pingRtt	rate	rateMin	rateMax	retransPkts%
☐	185.0	1s2cmid.vs	TCP_STREAM	kerneltest002+...	kerneltest001	1.0	bbr	60+...	0+...	8876.5	10	12263.0	11013.5	9535.0	4715.0	4820.0	1.69
☐	185.1	1s2cmid.vs	TCP_STREAM	kerneltest002	kerneltest001	1.0	bbr	60.0	0.0	8631.0	10	11403.0	10587.0	4715.0	4715.0	4715.0	
☐	185.2	1s2cmid.vs	TCP_STREAM	kerneltest004	kerneltest001	1.0	bbr	20.0	23.0	9122.0	10	13123.0	11440.0	4820.0	4820.0	4820.0	



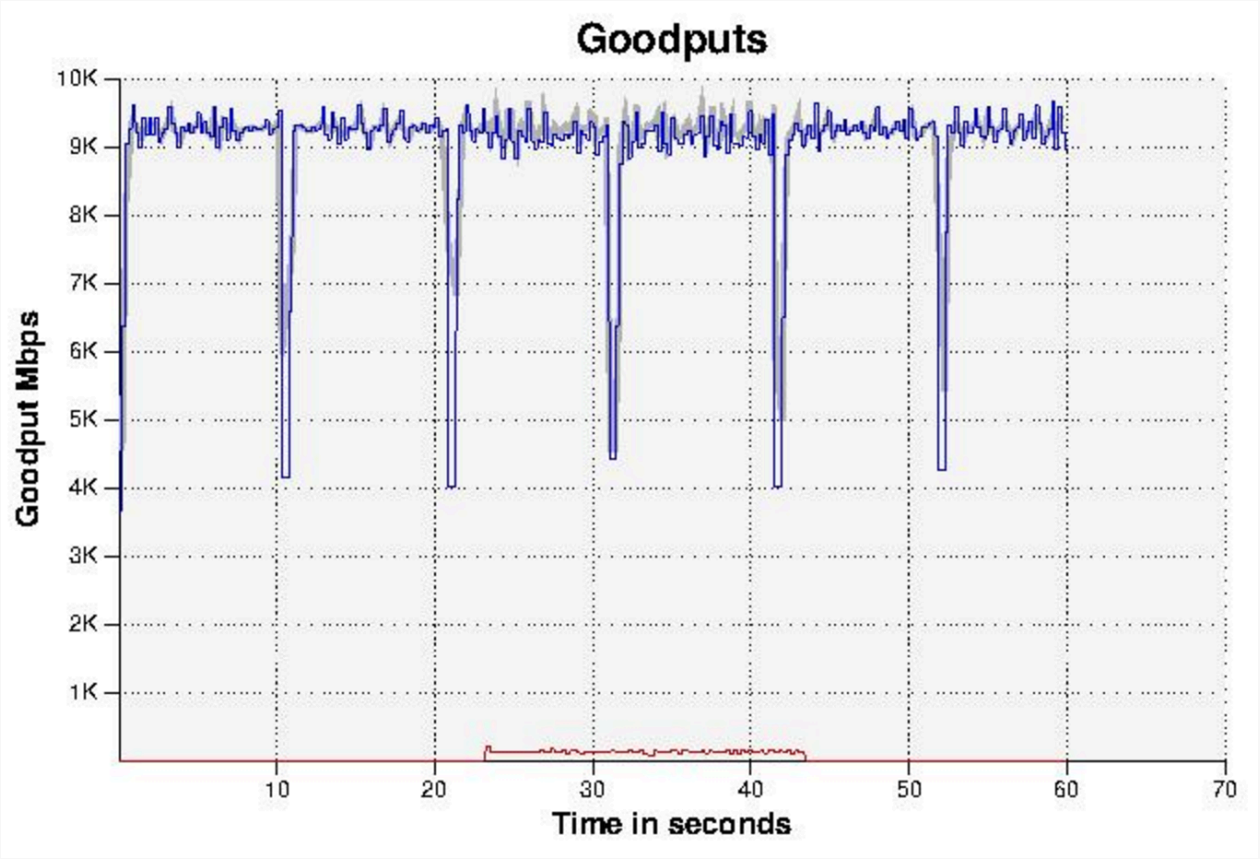


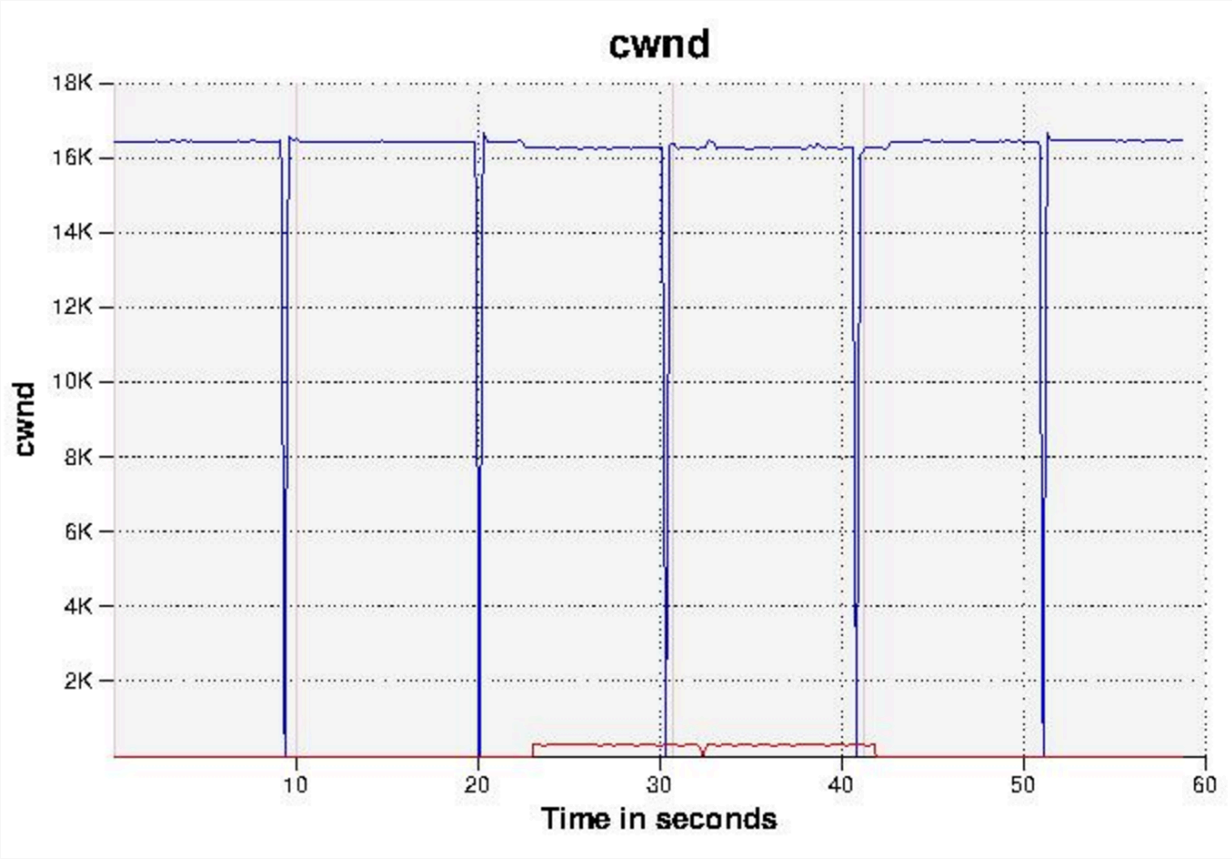

Goodputs
Goodput Mbps
11K
10K
9K
8K
7K
6K
5K
4K
3K
2K
1K
10
20
30
40
50
60
70
Time in seconds


cwnd
18K
16K
14K
12K
10K
8K
6K
4K
2K
cwnd
10
20
30
40
50
60
Time in seconds

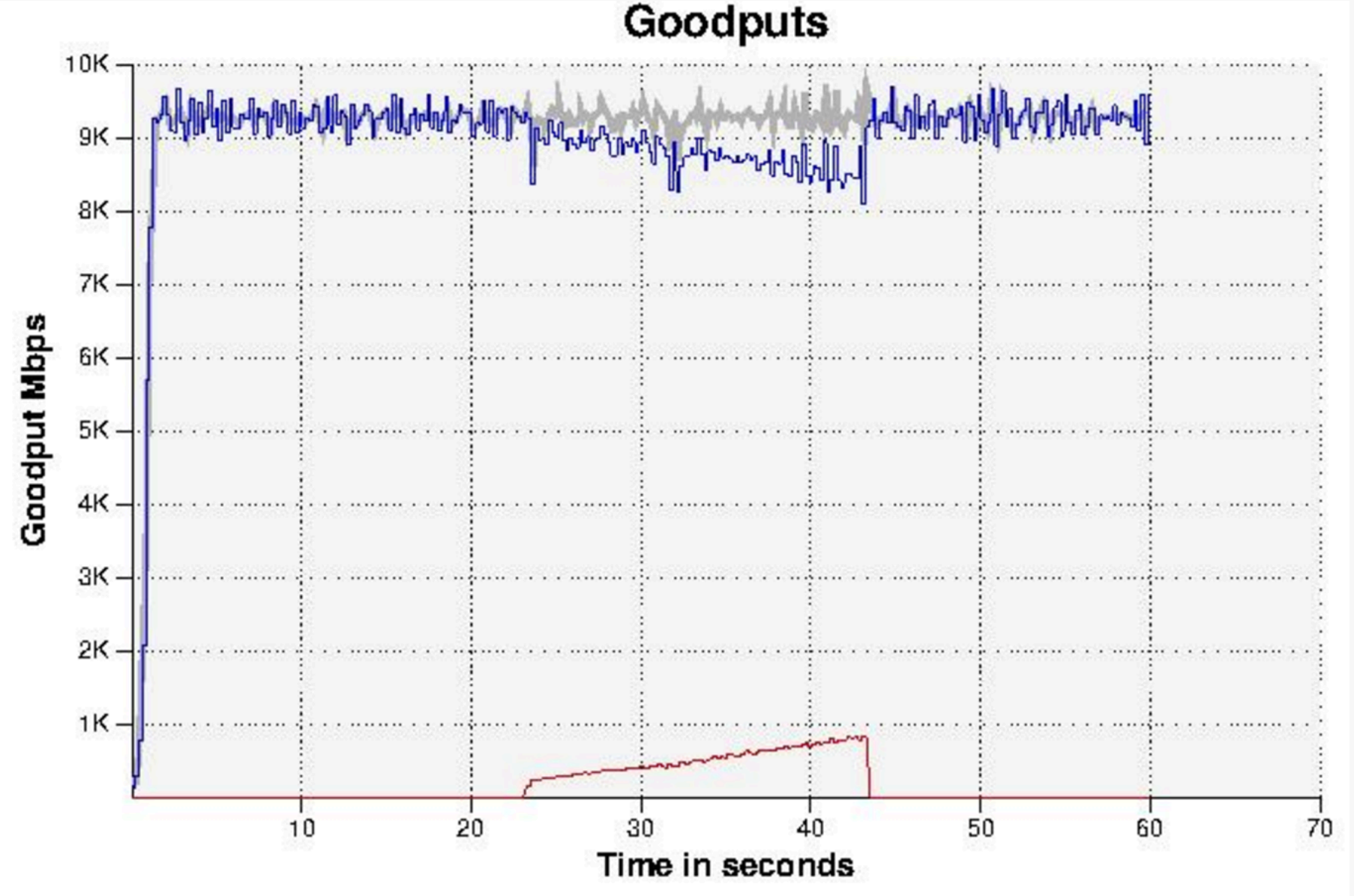
ANOTHER RUN, SAME TEST (WHY MACRO,REPS)

	exp	expName	test	host	server	instances	ca	dur	delay	cwnd	netem	rtt	rate	rateMin	rateMax	retransPkts%
☐	187.0	1s2cmid.vs	TCP_STREAM	kerneltest002+...	kerneltest001	1.0	bbr	60+...	0+...	8191.0	10	10873.0	9094.0	124.0	8970.0	0.03
☐	187.1	1s2cmid.vs	TCP_STREAM	kerneltest002	kerneltest001	1.0	bbr	60.0	0.0	16085.0	10	10623.0	8970.0	8970.0	8970.0	
☐	187.2	1s2cmid.vs	TCP_STREAM	kerneltest004	kerneltest001	1.0	bbr	20.0	23.0	297.0	10	11123.0	124.0	124.0	124.0	





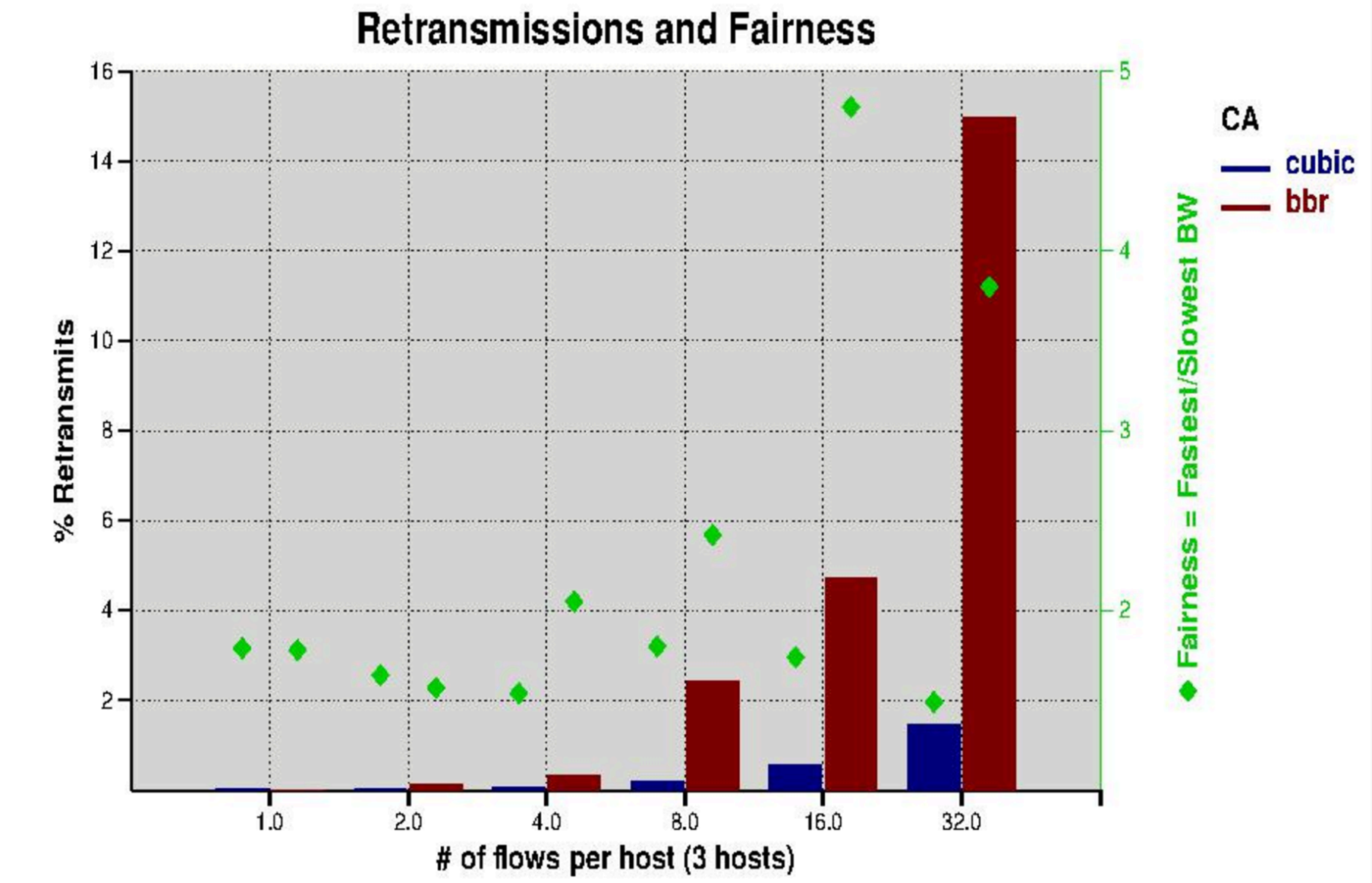
SAME TEST (2-FLOW) CUBIC



EXP.PY VISUALIZATION PROGRAM SAMPLE OUTPUT (1)

exp	expName	test	testType	instances	ca	cwnd	netem	bw	pingRtt	rate	rateMin	rateMax	retransPkts%	fairness
1514.0	1s3c01fs	TCP_STREAM	3Stream1	1.0	cubic	319.7	10	100mbit	114382.7	94.7	24.9	44.6	0.03	1.79
1515.0	1s3c02fs	TCP_STREAM	3Stream1	2.0	cubic	161.2	10	100mbit	114798.0	94.9	12.9	21.2	0.05	1.64
1516.0	1s3c04fs	TCP_STREAM	3Stream1	4.0	cubic	83.7	10	100mbit	118488.0	94.8	6.1	9.4	0.07	1.54
1517.0	1s3c08fs	TCP_STREAM	3Stream1	8.0	cubic	43.3	10	100mbit	120931.7	94.8	3.0	5.4	0.19	1.8
1518.0	1s3c16fs	TCP_STREAM	3Stream1	16.0	cubic	22.5	10	100mbit	123495.7	94.4	1.6	2.8	0.56	1.74
1519.0	1s3c32fs	TCP_STREAM	3Stream1	32.0	cubic	11.9	10	100mbit	126540.7	93.6	0.8	1.2	1.47	1.49
1520.0	1s3c01fs	TCP_STREAM	3Stream1	1.0	bbr	90.0	10	100mbit	28615.0	93.7	22.2	39.7	0.00	1.78
1521.0	1s3c02fs	TCP_STREAM	3Stream1	2.0	bbr	67.2	10	100mbit	44702.0	94.5	12.3	19.4	0.13	1.57
1522.0	1s3c04fs	TCP_STREAM	3Stream1	4.0	bbr	38.9	10	100mbit	51632.3	94.8	5.9	12.1	0.34	2.05
1523.0	1s3c08fs	TCP_STREAM	3Stream1	8.0	bbr	31.0	10	100mbit	73111.3	95.3	2.8	6.8	2.45	2.42
1524.0	1s3c16fs	TCP_STREAM	3Stream1	16.0	bbr	24.0	10	100mbit	92563.3	95.3	1.0	4.8	4.73	4.8
1525.0	1s3c32fs	TCP_STREAM	3Stream1	32.0	bbr	19.0	10	100mbit	127315.0	94.9	0.5	1.9	14.96	3.8

EXP.PY VISUALIZATION PROGRAM SAMPLE OUTPUT (2)



SAMPLE EXP.PY SCRIPT

```
File=bbr/exp
Path=.
relPath=bbr
IterFile=exp.html

# Select rows to use
Select=exp[has].0,testType==3Stream1,netem==10,bw==100mbit

# Create new fairness column
NewColField1=rateMax
NewColOp=/
NewColField2=rateMin
NewColName=fairness

# Select columns to use
Cols=exp,expName,test,testType,instances,ca,cwnd,netem,bw,pingRtt,rate,rateMin,rateMax,retransPkts%,fairness

# Write Table
doTable

# Write Plot
xUniform=on
#Y1Units=1=us,1000=ms,1000000=secs
plotX=instances
plotY1='retransPkts%'
plotY2='fairness'
plotSeries='ca'
plotXTitle='Flows per host (3 hosts)'
plotY1Title='% Retransmits'
plotY2Title='Fairness = Fastest/Slowest BW'
plotSeriesTitle='CA'
plotTitle='Retransmissions and Fairness'
plotXSize=1300
plotYSize=650
doPlot=plotRate
```

OTHER NETESTO SCRIPT COMMANDS

- SET_NETEM host=<host> delay=<delay in ms>
- DO_TCPDUMP host=<hostname> server=<servername> packets=<# packets>
- SET_QDISC host=<hostname> qdisc=<qdisc> rate=<rate> burst=<burst> limit=<limit>
- SET_SYSCTL host=<host> <name>=<val(s)>

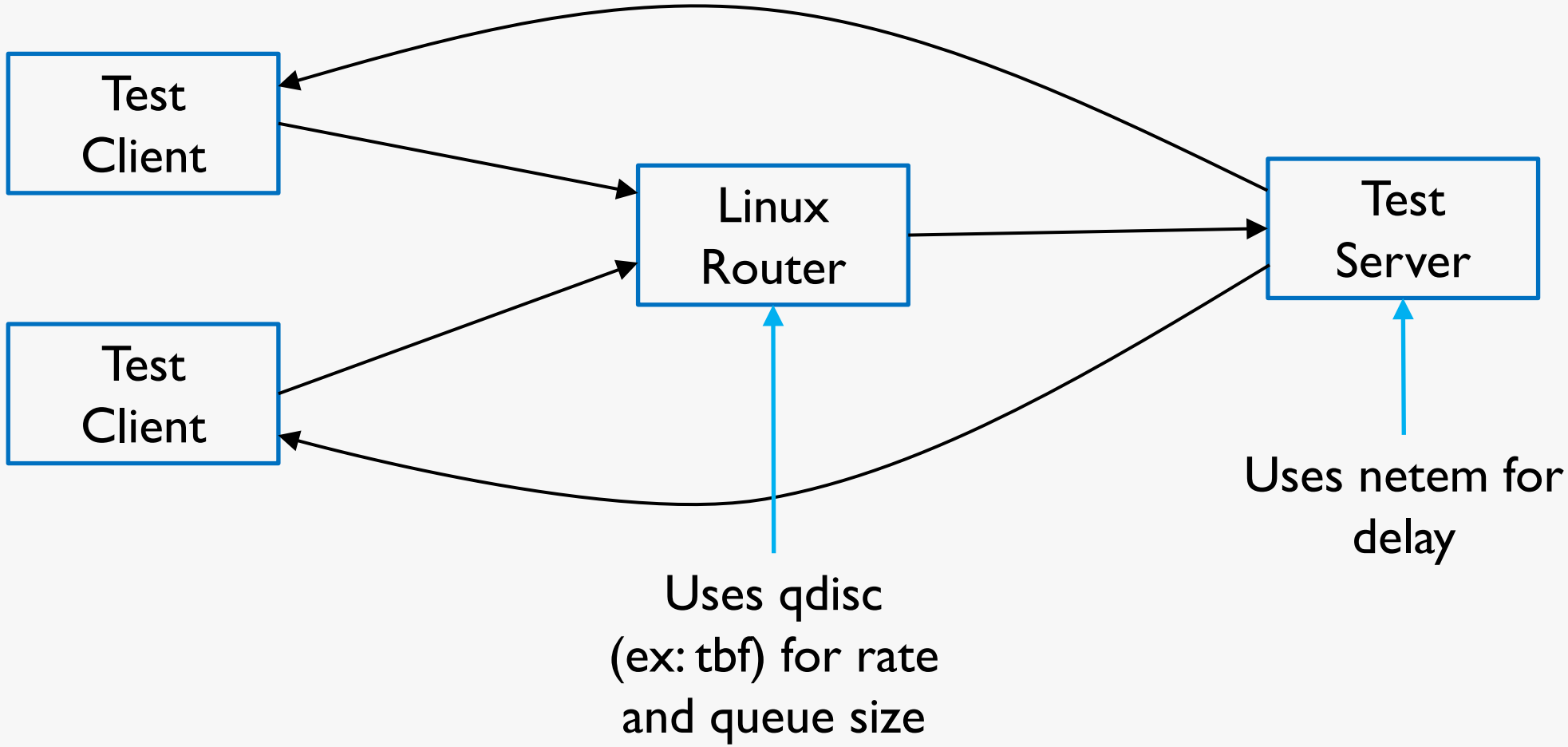
SAMPLE OF NETESTO MACROS

- **MServerRR** – Multiple server, multiple client TCP_RR. Each client sends to each server for each request size
  - Arguments: exp, servers, clients, expName, ca, dur, instances, reqs, reply
  - Ex: servers=host1,host2 clients=host3,host4,host5 reqs=10K,100K,1M
- **MServerStream** – Similar, but TCP_STREAM
- **MServerRRvs** - Two sets of clients, each with its own tcp ca
- Arguments: exp, servers, clients1, clients2, expName,ca1, ca2, dur, instances, reqs, reply

TCPCA23 MACRO

- Given baseCA and testCA will run 8 tests
  - more if len(testCA) > 1. Ex: SET testCA=reno,bbr
- 2&3 flow using base CA
- 2&3 flow using testCA
- 2&3 flow using baseCA for 1st flow and testCA for other flows
- 2&3 flow using testCA for 1st flow and baseCA for other flows
  - Last 2 examine fairness between CAs

TEST SETUP WITH LINUX ROUTER

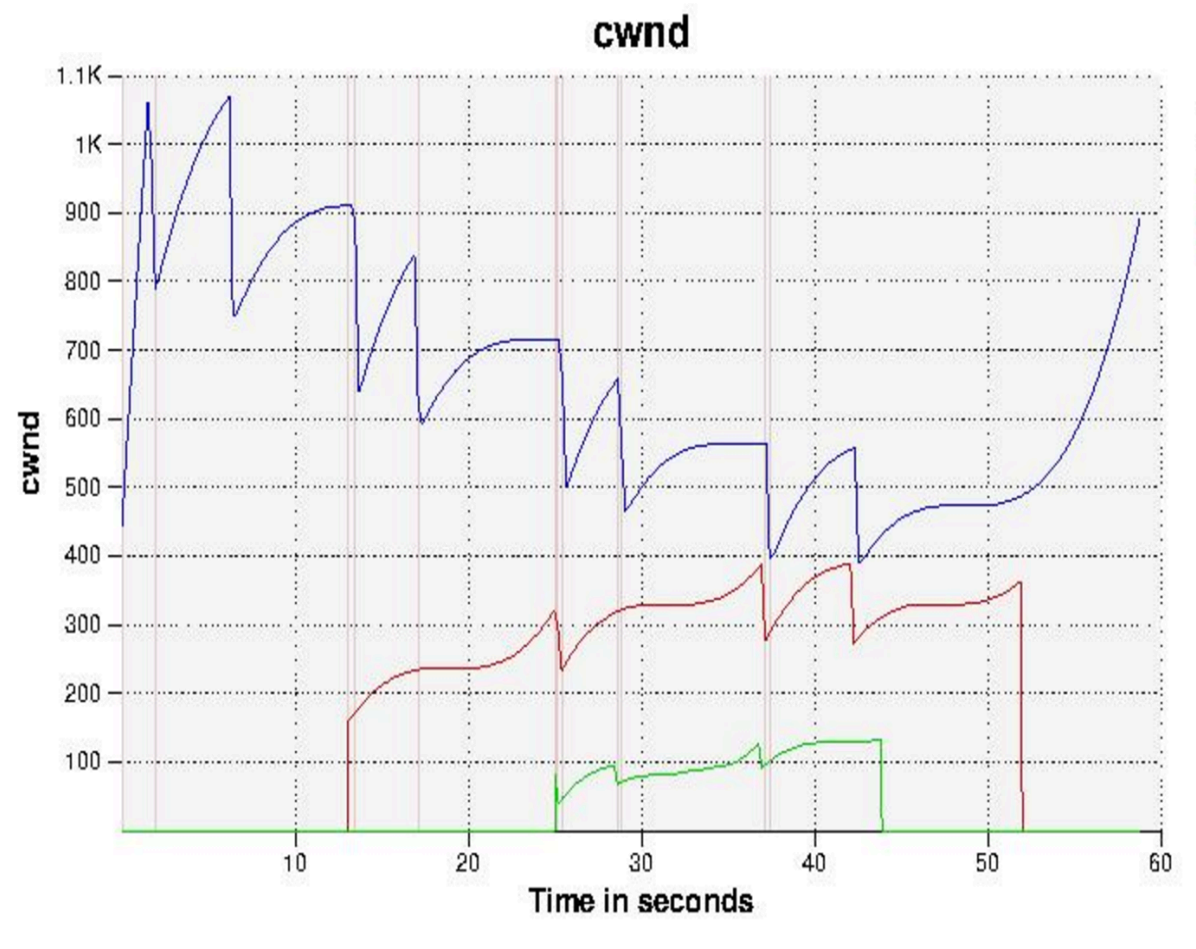


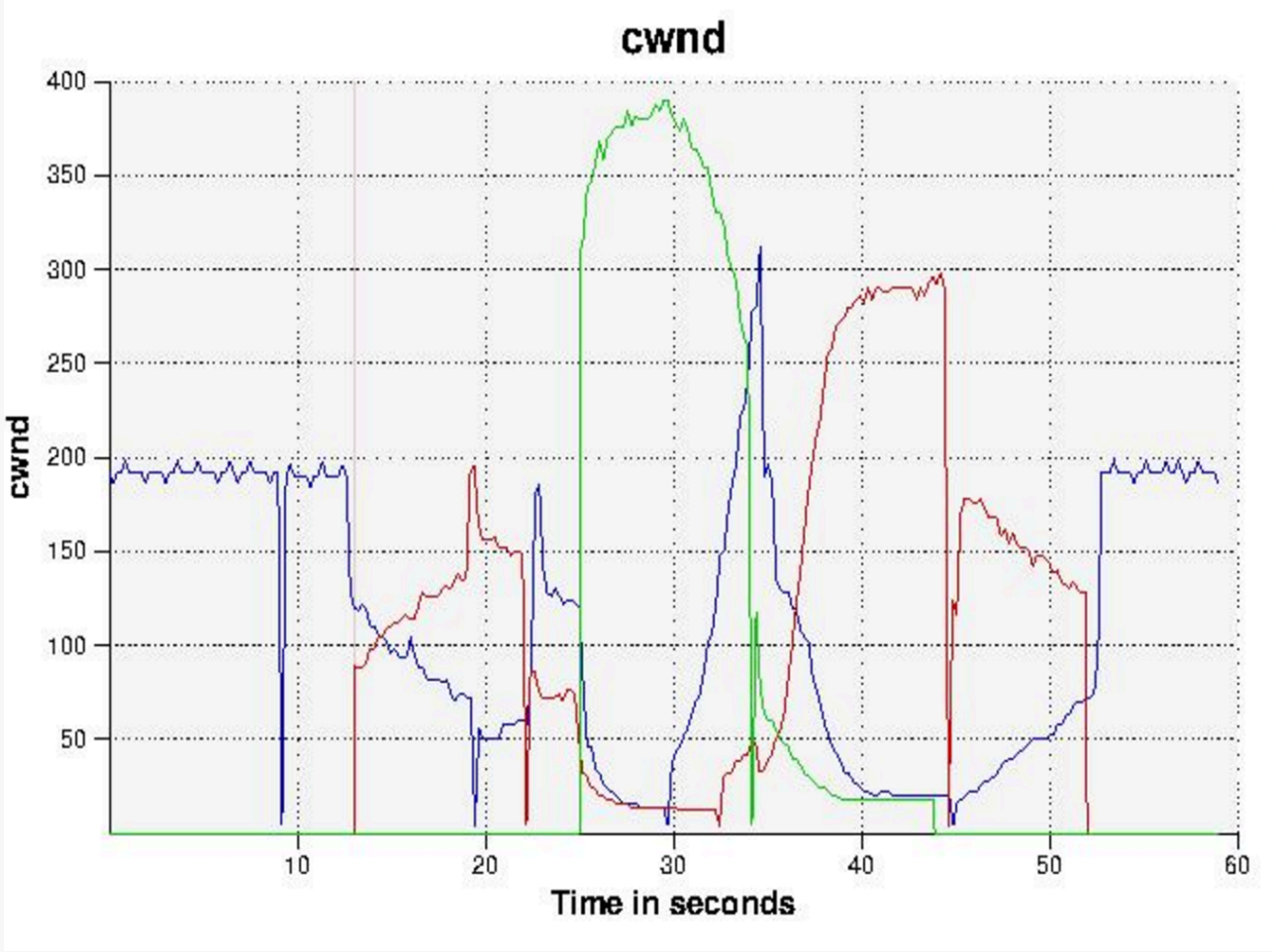
CAN CREATE MORE COMPLEX SCENARIOS

```
SET baselineCA=cubic
SET testCA=bbr              # could also do testCA=bbr,reno
SET delays=10,40
SET rates=5mbit,10mbit,40mbit,100mbit
SET limit=1500000  # in bytes, divide by 1500 for packets
SET_EXP buffers=$limit/1500

FOR delay IN $delays DO
  FOR rate IN $rates DO
    SET_QDISC host=$router qdisc=tbf rate=$rate burst=3000 limit=$limit
    RUN tcpCA23   # Could also do RUN,20 tcpCA23
  DONE
DONE
```

3-FLOW TESTS 100MBPS AND 10MS RTT





CONCLUSIONS

- Netesto makes it simple to run complex test scenarios
- Netesto makes simple to get a lot of information form the tests
- Netesto automatically creates HTML output for each test
- Netesto can create complex test summaries with tables and graphs
- Goal to create libraries of tests and associated exp scripts that can be shared among the community

AVAILABILITY

- By end of next week (4/14/17) at https://github.com/facebook/fbkutils

- Please wait until then, current version has some bugs